

```

# ----- Function -----
check_required_packages <- function(packages) {
  for (pkg in packages) {
    if (!requireNamespace(pkg, quietly = TRUE)) {
      stop(paste0("Package '", pkg, "' is required. Please install it first."))
    }
  }
}

load_housing_data <- function(file_path) {
  df <- read.csv(file_path, stringsAsFactors = FALSE)

  required_cols <- c("date", "hpi")
  missing_cols <- setdiff(required_cols, names(df))

  if (length(missing_cols) > 0) {
    stop(paste("Missing required columns:", paste(missing_cols, collapse = ",
")))
  }

  df$date <- as.Date(df$date)
  df <- df[order(df$date), ]

  if (!"log_hpi" %in% names(df)) {
    df$log_hpi <- log(df$hpi)
  }

  if (!"diff_log_hpi" %in% names(df)) {
    df$diff_log_hpi <- c(NA, diff(df$log_hpi))
  }

  return(df)
}

make_monthly_ts <- function(values, dates) {
  start_year <- as.integer(format(min(dates), "%Y"))
  start_month <- as.integer(format(min(dates), "%m"))

  ts(
    values,
    start = c(start_year, start_month),
    frequency = 12
  )
}

add_dummy_variables <- function(df) {
  df$post2002 <- ifelse(df$date >= as.Date("2002-01-01"), 1, 0)

  df$crisis <- ifelse(
    df$date >= as.Date("2007-01-01") &
      df$date <= as.Date("2010-12-01"),
    1,
    0
  )
}

```

```

    return(df)
}

prepare_model_inputs <- function(df) {
  df <- add_dummy_variables(df)

  log_hpi_ts <- make_monthly_ts(
    values = df$log_hpi,
    dates = df$date
  )

  df_diff <- df[!is.na(df$diff_log_hpi), ]

  diff_log_hpi_ts <- make_monthly_ts(
    values = df_diff$diff_log_hpi,
    dates = df_diff$date
  )

  xreg_crisis <- as.matrix(df_diff[, c("crisis"), drop = FALSE])
  xreg_both <- as.matrix(df_diff[, c("post2002", "crisis"), drop = FALSE])

  list(
    df = df,
    df_diff = df_diff,
    log_hpi_ts = log_hpi_ts,
    diff_log_hpi_ts = diff_log_hpi_ts,
    xreg_crisis = xreg_crisis,
    xreg_both = xreg_both
  )
}

fit_candidate_models <- function(inputs) {
  models <- list(
    "Random Walk with Drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(0, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),

    "ARIMA(1,1,0) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 0),
      include.drift = TRUE,
      method = "ML"
    ),

    "ARIMA(1,1,1) with drift" = forecast::Arima(
      inputs$log_hpi_ts,
      order = c(1, 1, 1),
      include.drift = TRUE,
      method = "ML"
    ),
  )
}

```

```

"ARIMA(2,1,1) with drift" = forecast::Arima(
  inputs$log_hpi_ts,
  order = c(2, 1, 1),
  include.drift = TRUE,
  method = "ML"
),

"SARIMA(1,1,1)(1,0,0)[12] with drift" = forecast::Arima(
  inputs$log_hpi_ts,
  order = c(1, 1, 1),
  seasonal = list(order = c(1, 0, 0), period = 12),
  include.drift = TRUE,
  method = "ML"
),

"SARIMA(1,1,1)(0,0,1)[12] with drift" = forecast::Arima(
  inputs$log_hpi_ts,
  order = c(1, 1, 1),
  seasonal = list(order = c(0, 0, 1), period = 12),
  include.drift = TRUE,
  method = "ML"
),

"ARIMAX(1,1,1) with crisis dummy" = forecast::Arima(
  inputs$diff_log_hpi_ts,
  order = c(1, 0, 1),
  xreg = inputs$xreg_crisis,
  include.mean = TRUE,
  method = "ML"
),

"ARIMAX(1,1,1) with post-2002 dummy and crisis dummy" = forecast::Arima(
  inputs$diff_log_hpi_ts,
  order = c(1, 0, 1),
  xreg = inputs$xreg_both,
  include.mean = TRUE,
  method = "ML"
),

"SARIMAX(1,1,1)(1,0,0)[12] with crisis dummy" = forecast::Arima(
  inputs$diff_log_hpi_ts,
  order = c(1, 0, 1),
  seasonal = list(order = c(1, 0, 0), period = 12),
  xreg = inputs$xreg_crisis,
  include.mean = TRUE,
  method = "ML"
),

"SARIMAX(1,1,1)(1,0,0)[12] with post-2002 dummy and crisis dummy" =
forecast::Arima(
  inputs$diff_log_hpi_ts,
  order = c(1, 0, 1),
  seasonal = list(order = c(1, 0, 0), period = 12),
  xreg = inputs$xreg_both,
  include.mean = TRUE,

```

```

        method = "ML"
    )
}

return(models)
}

run_model_estimation <- function(file_path) {
  set.seed(123)

  check_required_packages(c("forecast"))

  df <- load_housing_data(file_path)
  inputs <- prepare_model_inputs(df)
  models <- fit_candidate_models(inputs)

  list(
    data = df,
    inputs = inputs,
    models = models
  )
}

get_clean_residuals <- function(fit) {
  residuals <- stats::residuals(fit)
  residuals <- residuals[!is.na(residuals)]
  return(residuals)
}

get_max_abs_residual_acf <- function(fit, lag.max = 24) {
  residuals <- get_clean_residuals(fit)

  acf_values <- stats::acf(
    residuals,
    lag.max = lag.max,
    plot = FALSE
  )$acf

  acf_values <- acf_values[-1]

  max(abs(acf_values))
}

run_ljung_box_test <- function(fit, lag = 24) {
  residuals <- get_clean_residuals(fit)
  n_parameters <- length(stats::coef(fit))

  test_result <- stats::Box.test(
    residuals,
    lag = lag,
    type = "Ljung-Box",
    fitdf = n_parameters
  )

  list(

```

```

    statistic = as.numeric(test_result$statistic),
    p_value = as.numeric(test_result$p.value)
  )
}

diagnose_single_model <- function(model_name, fit, lag = 24, acf_lag_max = 24)
{
  residuals <- get_clean_residuals(fit)
  ljung_box <- run_ljung_box_test(fit, lag = lag)
  max_abs_acf <- get_max_abs_residual_acf(fit, lag.max = acf_lag_max)

  data.frame(
    Model = model_name,
    Residual_Mean = mean(residuals),
    Residual_SD = stats::sd(residuals),
    Max_Abs_ACF_Lag_1_to_24 = max_abs_acf,
    Ljung_Box_Lag = lag,
    Ljung_Box_Statistic = ljung_box$statistic,
    Ljung_Box_p_value = ljung_box$p_value,
    row.names = NULL
  )
}

diagnose_all_models <- function(models, lag = 24, acf_lag_max = 24) {
  diagnostics_table <- do.call(
    rbind,
    lapply(names(models), function(model_name) {
      diagnose_single_model(
        model_name = model_name,
        fit = models[[model_name]],
        lag = lag,
        acf_lag_max = acf_lag_max
      )
    })
  )

  diagnostics_table <- diagnostics_table[order(
    diagnostics_table$Ljung_Box_Statistic,
    diagnostics_table$Max_Abs_ACF_Lag_1_to_24
  ), ]

  return(diagnostics_table)
}

select_models_by_relative_diagnostics <- function(diagnostics_table, top_n = 4)
{
  head(diagnostics_table$Model, top_n)
}

add_retention_decision <- function(diagnostics_table, selected_model_names) {
  diagnostics_table$Diagnostics_Decision <- ifelse(
    diagnostics_table$Model %in% selected_model_names,
    "Retained for forecasting",
    "Not retained"
  )
}

```

```

    return(diagnostics_table)
}

# Visualization: all-model comparison plots
plot_ljung_box_p_values <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  ordered_table <-
diagnostics_table[order(diagnostics_table$Ljung_Box_p_value), ]

  par(mfrow = c(1, 1))
  par(mar = c(5, 16, 4, 2))

  barplot(
    ordered_table$Ljung_Box_p_value,
    names.arg = ordered_table$Model,
    horiz = TRUE,
    las = 1,
    xlab = "Ljung-Box p-value",
    main = "Residual Diagnostics: Ljung-Box p-values"
  )

  abline(v = 0.05, lty = 2)
}

plot_ljung_box_statistics <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  ordered_table <-
diagnostics_table[order(diagnostics_table$Ljung_Box_Statistic), ]

  par(mfrow = c(1, 1))
  par(mar = c(5, 16, 4, 2))

  barplot(
    ordered_table$Ljung_Box_Statistic,
    names.arg = ordered_table$Model,
    horiz = TRUE,
    las = 1,
    xlab = "Ljung-Box statistic",
    main = "Residual Diagnostics: Ljung-Box Statistics"
  )
}

plot_max_abs_residual_acf <- function(diagnostics_table) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

```

```

ordered_table <-
diagnostics_table[order(diagnostics_table$Max_Abs_ACF_Lag_1_to_24), ]

par(mfrow = c(1, 1))
par(mar = c(5, 16, 4, 2))

barplot(
  ordered_table$Max_Abs_ACF_Lag_1_to_24,
  names.arg = ordered_table$Model,
  horiz = TRUE,
  las = 1,
  xlab = "Maximum absolute residual ACF",
  main = "Residual Diagnostics: Maximum Absolute Residual ACF"
)
}

# Visualization: individual residual diagnostics
plot_residual_time_series <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

  plot(
    residuals,
    type = "l",
    main = paste("Residual Plot:", model_name),
    xlab = "Time Index",
    ylab = "Residuals"
  )

  abline(h = 0, lty = 2)
}

plot_residual_acf <- function(fit, model_name, lag.max = 48) {
  residuals <- get_clean_residuals(fit)

  stats::acf(
    residuals,
    lag.max = lag.max,
    main = paste("ACF of Residuals:", model_name)
  )
}

plot_residual_histogram <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

  hist(
    residuals,
    breaks = 20,
    main = paste("Histogram:", model_name),
    xlab = "Residuals"
  )
}

plot_residual_qq <- function(fit, model_name) {
  residuals <- get_clean_residuals(fit)

```

```

qqnorm(
  residuals,
  main = paste("Q-Q Plot:", model_name)
)

qqline(residuals)
}

plot_full_residual_diagnostics <- function(fit, model_name) {
  old_mfrow <- par("mfrow")
  old_mar <- par("mar")
  on.exit(par(mfrow = old_mfrow, mar = old_mar))

  par(mfrow = c(2, 2))
  par(mar = c(4, 4, 3, 1))

  plot_residual_time_series(fit, model_name)
  plot_residual_acf(fit, model_name)
  plot_residual_histogram(fit, model_name)
  plot_residual_qq(fit, model_name)
}

plot_all_model_diagnostics <- function(models) {
  for (model_name in names(models)) {
    cat("\nPlotting residual diagnostics for:", model_name, "\n")

    plot_full_residual_diagnostics(
      fit = models[[model_name]],
      model_name = model_name
    )

    readline(prompt = "Press [Enter] to continue to the next model...")
  }
}

# Main diagnostics runner
run_model_diagnostics <- function(results, top_n = 4, lag = 24) {
  models <- results$models

  diagnostics_table <- diagnose_all_models(
    models = models,
    lag = lag,
    acf_lag_max = lag
  )

  selected_model_names <- select_models_by_relative_diagnostics(
    diagnostics_table = diagnostics_table,
    top_n = top_n
  )

  diagnostics_table <- add_retention_decision(
    diagnostics_table = diagnostics_table,
    selected_model_names = selected_model_names
  )
}

```

```

cat("\n===== Residual Diagnostics Table =====\n")
print(diagnostics_table, row.names = FALSE, digits = 5)

cat("\n===== Models Retained for Forecasting =====\n")
print(selected_model_names)

cat("\nAll models are ranked by smaller Ljung-Box statistic first, then
smaller maximum absolute residual ACF.\n")

# Visualize all-model comparison before selecting models.
plot_ljung_box_p_values(diagnostics_table)
readline(prompt = "Press [Enter] to continue to Ljung-Box statistic plot...")

plot_ljung_box_statistics(diagnostics_table)
readline(prompt = "Press [Enter] to continue to Max ACF plot...")

plot_max_abs_residual_acf(diagnostics_table)
readline(prompt = "Press [Enter] to continue to individual residual
diagnostics for all models...")

# Visualize every candidate model.
plot_all_model_diagnostics(models)

list(
  diagnostics_table = diagnostics_table,
  selected_model_names = selected_model_names
)
}

# ----- Main -----
set.seed(123)
results <- run_model_estimation(
  file_path = "C:/Users/use/Desktop/NCCU_STAT/Time
Series/final/data/processed/monthly_housing_train.csv"
)

diagnostics_results <- run_model_diagnostics(
  results = results,
  top_n = 4,
  lag = 24
)

```